

Meta-Optimization of a Real-Coded Genetic Algorithm for Continuous Benchmark Functions

Negura Teodor Alexandru “University A.I. Cuza”

The Second Assignment – Extended Study

Abstract

This report extends the work from Homework 2 by introducing a meta-optimization layer on top of the real-coded Genetic Algorithm (GA) previously designed for continuous function minimization. The goal of this extended study is to automatically tune the main GA parameters (population size, crossover probability, mutation probability and the distribution indices of SBX and polynomial mutation) via a higher-level *meta-GA*, and to quantify the performance gain over the fixed-parameter GA from Homework 2.

The study considers the same four benchmark functions as before—De Jong 1, Rastrigin, Schwefel and Michalewicz—in dimensions $d \in \{5, 10, 30\}$. For each (function, d) pair, a dedicated meta-GA searches in the parameter space of the underlying GA. The best configuration found by the meta-GA is then re-evaluated in 30 independent runs and compared to the Homework 2 baseline in terms of solution quality (best, mean, standard deviation) and relative improvement.

The results show that for functions where the baseline GA already reaches machine precision (De Jong 1 and low-dimensional Rastrigin), meta-optimization cannot bring substantial gains. In contrast, on more challenging landscapes (Schwefel and Michalewicz) the meta-optimized GA achieves consistent improvements of up to about 13% in mean fitness, while also reducing variance in most cases.

1 Introduction

Continuous black-box optimization appears in many scientific and engineering applications: parameter tuning of physical models, machine learning, signal processing, control, and many others. Often the objective function is non-convex, noisy or non-differentiable, making gradient-based methods inadequate or highly sensitive to initialization.

In this context, metaheuristics such as hill climbing, Simulated Annealing (SA) and Genetic Algorithms (GAs) provide robust, general-purpose tools for global optimization. Homework 1 focused on local-search methods (several hill-climbing variants and SA), while Homework 2 moved to a population-based approach and designed a real-coded GA capable of reaching very high precision on four standard benchmark functions in up to 30 dimensions.

Although the Homework 2 GA already outperformed the local-search baselines by a large margin, its performance still depended strongly on manually chosen parameters (population size, crossover and mutation rates, distribution indices of variation operators). Choosing these parameters by hand is time-consuming and problem-dependent, and there is no guarantee that they are close to optimal.

This extended study addresses this limitation by introducing a *meta-GA*: a second-level genetic algorithm that searches in the parameter space of the original GA. The objectives of this report are:

1. to define a suitable parameterization and search space for the real-coded GA from Homework 2;

2. to design and implement a meta-GA that automatically tunes these parameters on each benchmark function and dimensionality;
3. to compare quantitatively the performance of the meta-optimized GA against the fixed-parameter GA of Homework 2.

2 Methods

2.1 Benchmark functions and problem formulation

We consider the same four continuous benchmark functions used in Homework 2.

De Jong 1 (Sphere).

$$f_{\text{DJ1}}(x) = \sum_{i=1}^d x_i^2, \quad (1)$$

with $x_i \in [-5.12, 5.12]$ and global minimum $f^* = 0$ at $x^* = 0$. This is a smooth, strictly convex and unimodal function.

Rastrigin.

$$f_{\text{Ras}}(x) = 10d + \sum_{i=1}^d (x_i^2 - 10 \cos(2\pi x_i)), \quad (2)$$

with $x_i \in [-5.12, 5.12]$ and global minimum $f^* = 0$ at $x^* = 0$. The function is highly multimodal, with a large number of regularly spaced local minima.

Schwefel. We use the classical Schwefel function with domain $x_i \in [-500, 500]$:

$$f_{\text{Sch}}(x) = 418.9829 d - \sum_{i=1}^d x_i \sin(\sqrt{|x_i|}), \quad (3)$$

which is deceptive: the global optimum is far from the origin, while many deep local minima are closer.

Michalewicz.

$$f_{\text{Mich}}(x) = - \sum_{i=1}^d \sin(x_i) \left[\sin\left(\frac{ix_i^2}{\pi}\right) \right]^{2m}, \quad (4)$$

with $m = 10$ and $x_i \in [0, \pi]$. The function has sharp, narrow valleys and is difficult in high dimension.

In all cases, the optimization task is

$$\min_{x \in D \subset \mathbb{R}^d} f(x), \quad (5)$$

for $d \in \{5, 10, 30\}$, as in Homework 2.

2.2 Baseline real-coded GA from Homework 2

The baseline algorithm is the real-coded GA designed in Homework 2. Only a brief summary is given here, since the full details are already documented.¹

Representation and initialization. Each individual encodes a candidate solution $x = (x_1, \dots, x_d) \in \mathbb{R}^d$ as a vector of doubles. This real-coded representation avoids the precision and Hamming-cliff issues associated with binary encodings.

The initial population is generated by sampling each coordinate independently from a uniform distribution over the domain of the corresponding function, e.g. $x_i \sim U[-5.12, 5.12]$ for De Jong 1 and Rastrigin. This ensures that the initial population covers the whole search space and does not introduce a bias towards any particular region.

Fitness evaluation. All four benchmarks are minimization problems. The GA uses the raw function value as fitness and interprets lower values as better; no additional rescaling is applied.

Selection: tournament selection. Parent selection uses k -tournament selection with $k = 3$:

1. sample k individuals uniformly at random from the population;
2. choose the individual with the lowest fitness as the parent.

Tournament selection is robust and independent of absolute fitness values, making it suitable for functions with negative optima (e.g. Michalewicz, Schwefel). By tuning k , the selection pressure can be controlled; in our experiments $k = 3$ offered a good balance between exploration and exploitation.

Crossover: Simulated Binary Crossover (SBX). For recombination we use Simulated Binary Crossover (SBX), a standard operator for real-coded GAs:

- for each pair of parents $(x^{(1)}, x^{(2)})$ and each gene i , SBX generates two children $y_i^{(1)}$ and $y_i^{(2)}$ by sampling around the parents according to a distribution controlled by a parameter η_c (the distribution index);
- when parents are close in a coordinate, children are also close, which favours exploitation; when parents are far apart, SBX explores the space between them.

We used a crossover probability $p_c = 0.9$ and a distribution index $\eta_c = 15$. A high crossover rate, combined with tournament selection and elitism, allows good building blocks to be propagated through the population.

Mutation: polynomial mutation. Mutation uses a polynomial mutation operator, which for each gene perturbs x_i by a small value Δ drawn from a polynomial distribution controlled by a parameter η_m :

- with high probability, mutations are small and refine the current solution (exploitation);
- with small probability, mutations can be larger jumps, helping the algorithm escape local minima (exploration).

We used a per-gene mutation probability $p_m = 1/d$ and $\eta_m = 20$, which biases the search towards fine-tuning in later generations, while still allowing occasional larger moves in early generations.

¹and were guided by the course notes.

Replacement and elitism. Generational replacement is used: in each generation a full offspring population is created and replaces the current one, with *elitism*: the best individual of generation t is copied unchanged to generation $t + 1$. This guarantees that the best fitness found so far is monotonically non-increasing over generations and avoids the loss of good solutions due to stochastic effects.

Implementation details. The GA is implemented in C++ using an object-oriented architecture:

- an `ObjectiveFunctions` module encapsulates the definitions and domains of De Jong 1, Rastrigin, Schwefel and Michalewicz;
- a `GeneticAlgorithm` class manages the population, selection, variation, and the main evolutionary loop;
- an `Individual` structure stores the real-valued genotype and its fitness.

Random numbers are generated using the C++ standard library Mersenne Twister, with different seeds for each independent run. The code was instrumented to record, for every run, the best fitness per generation and the summary of the final best solution (best, worst, mean, standard deviation, mean error).

Baseline parameter settings. In Homework 2, the following population sizes N and number of generations T were used:

- $d = 5, 10$: $N = 150$, $T = 1500$;
- $d = 30$: $N = 300$, $T = 3000$;
- Schwefel in $d = 30$ (hardest case): $N = 1000$, $T = 5000$.

For each configuration (function, dimension), the GA was run 30 times with different random seeds and summary statistics of the final best fitness were reported.

2.3 Meta-Genetic Algorithm for parameter optimization

This extended study introduces a second-level GA—the *meta-GA*—which searches in the parameter space of the baseline GA. The inner GA (“base GA”) retains its structure (representation, operators), but its numerical parameters are no longer fixed by hand; instead they are encoded as genes of the meta-individuals.

Parameter vector and search space

Each meta-individual encodes a parameter vector

$$\theta = (N, p_m, p_c, \eta_c, \eta_m),$$

where:

- $N \in \{300, \dots, 1000\}$ is the population size (integer);
- $p_m \in [0.01, 0.40]$ is the per-gene mutation probability;
- $p_c \in [0.60, 0.95]$ is the crossover probability;

- $\eta_c \in [5, 40]$ is the SBX distribution index;
- $\eta_m \in [5, 40]$ is the polynomial mutation distribution index.

The bounds are chosen to cover a broad range around the values used in Homework 2, while avoiding obviously degenerate configurations.

In practice, N is encoded as a real variable and rounded to the closest integer when instantiating the base GA.

Fitness of a meta-individual

For a fixed benchmark function f and dimension d , the fitness of a meta-individual θ is obtained by running the base GA with parameters θ and recording the final best fitness after a fixed number of generations $T(d)$ (same T as in Homework 2 for that dimension). To reduce noise, the meta-fitness is defined as the mean best fitness over $R = 5$ independent base-GA runs with the same θ but different seeds:

$$F(\theta) = \frac{1}{R} \sum_{r=1}^R \min_{x \in D} f(x; \theta, r).$$

Since we minimize the benchmark functions, the meta-GA also minimizes $F(\theta)$.

Meta-GA operators and settings

The meta-GA itself is a real-coded GA with the following settings:

- population size $N_{\text{meta}} = 40$;
- tournament selection of size $k = 2$;
- SBX crossover with $p_c^{\text{meta}} = 0.9$ and $\eta_c^{\text{meta}} = 10$;
- polynomial mutation with per-gene rate $p_m^{\text{meta}} = 0.2$ and $\eta_m^{\text{meta}} = 15$;
- generational replacement with elitism (best meta-individual copied);
- number of meta-generations $T_{\text{meta}} = 50$.

A separate meta-GA is run for each pair (function, d). The best meta-individual θ^* at the end of the meta-run is interpreted as the *optimized parameter configuration* for that problem. These configurations are listed in Section 4.2, where they are compared against the baseline parameters and results from Homework 2.

3 Experimental setup

3.1 Dimensions, repetitions and stopping criteria

The experiments reuse the same dimensionalities as Homework 2: $d \in \{5, 10, 30\}$. For each function and dimension, we distinguish two phases:

1. **Meta-optimization phase.** The meta-GA is run on the chosen (function, d) until $T_{\text{meta}} = 50$ generations. Its output is the best parameter vector $\theta^* = (N, p_m, p_c, \eta_c, \eta_m)$.

2. **Evaluation phase.** The base GA is run with parameters θ^* for $T(d)$ generations (same T as in Homework 2) and evaluated over 30 independent runs. The same was done in Homework 2, but with fixed parameters instead of θ^* .

This allows a direct comparison between:

- the *baseline GA* (Homework 2) with fixed parameters;
- the *meta-optimized GA* (this extended study) with parameters tuned by the meta-GA.

3.2 Measured statistics

For both the baseline and the meta-optimized GA, and for every (function, dimension) pair, we report:

- best (minimum) fitness over 30 runs;
- mean and standard deviation of the final best fitness;
- worst (maximum) final best fitness;
- relative improvement in mean fitness:

$$\text{Improvement} = \frac{\text{Baseline mean} - \text{Optimized mean}}{|\text{Baseline mean}|} \times 100\%.$$

4 Experimental results

4.1 Baseline results from Homework 2

To provide context for the meta-optimization results, we first present the baseline performance of the fixed-parameter GA from Homework 2. Table 1 shows the results obtained with the manually tuned parameters: $p_c = 0.9$, $\eta_c = 15$, $p_m = 1/d$, $\eta_m = 20$, and the population sizes and generation counts specified in Section 2.2.

As Table 1 shows, the Homework 2 GA already achieved excellent results on De Jong 1 and Rastrigin, reaching machine precision in most cases. However, on Schwefel and Michalewicz there is still room for improvement, with noticeable variance across runs and mean fitness values that are not at the theoretical optimum.

4.2 Optimized parameter configurations

Table 2 summarizes the parameter configurations selected by the meta-GA for each benchmark function and dimensionality. For each configuration we report the population size N , mutation and crossover probabilities (p_m, p_c) , distribution indices (η_c, η_m) , and the average fitness used during the meta-optimization phase.

Several trends are visible already: population size tends to increase with dimensionality on De Jong 1 and Rastrigin, while for Schwefel the meta-GA prefers a moderately sized population already in 30D (compared to the very large hand-chosen population from Homework 2). Mutation rates are relatively high for 5D and 10D Schwefel and Michalewicz, encouraging exploration on these rugged landscapes, while in 30D they drop to small values combined with large distribution indices, favouring fine tuning near good basins.

Table 1: Baseline GA results from Homework 2 for each benchmark function and dimension (30 runs per configuration). These results serve as the baseline for evaluating the improvements achieved by meta-optimization.

Function	d	Best	Mean	Std. dev.	Worst	Mean Δ
De Jong 1	5	0.000000	0.000000	0.000000	0.000000	0.000000
De Jong 1	10	0.000000	0.000000	0.000000	0.000000	0.000000
De Jong 1	30	0.000000	0.000000	0.000000	0.000000	0.000000
Rastrigin	5	0.000000	0.000000	0.000000	0.000001	0.000000
Rastrigin	10	0.000000	0.000002	0.000002	0.000007	0.000002
Rastrigin	30	0.000018	0.000034	0.000012	0.000085	0.000034
Schwefel	5	-2094.914436	-1925.152823	121.106053	-1621.161098	169.761677
Schwefel	10	-4189.828868	-3759.502918	187.225829	-3479.198862	430.326082
Schwefel	30	-11621.979816	-11089.007358	279.858859	-10556.034850	1480.479642
Michalewicz	5	-4.687658	-4.686266	0.007497	-4.645895	0.002006
Michalewicz	10	-9.660152	-9.649157	0.031539	-9.497392	0.011094
Michalewicz	30	-29.615087	-29.355069	0.302678	-28.616124	0.275831

4.3 Performance comparison with the Homework 2 GA

Table 3 reports the detailed comparison between the baseline GA (Homework 2, results from Table 1) and the meta-optimized GA (this extended study, using parameters from Table 2) on all functions and dimensions. For each configuration we show the baseline mean fitness, and the best, mean, standard deviation and relative improvement achieved by the meta-optimized GA.

De Jong 1. On the unimodal De Jong 1 function, the Homework 2 GA already reached the global optimum to machine precision in all dimensions, with zero variance. Consequently, the meta-GA cannot find any parameter configuration that improves upon a mean of exactly zero; all statistics remain at numerical zero for both the baseline and the optimized GA. This confirms the correctness and robustness of the original implementation.

Rastrigin. For Rastrigin, the baseline GA was already extremely accurate, but in 10D and 30D the mean fitness remained slightly above zero (2×10^{-6} and 3.4×10^{-5} , respectively). After meta-optimization, the GA consistently finds solutions with mean fitness numerically equal to zero and zero standard deviation in 30 runs. In absolute terms, the improvement is tiny, but it shows that the meta-GA is capable of fine-tuning the parameters enough to completely eliminate residual numerical error on a highly multimodal landscape.

Schwefel. The most significant gains appear on the deceptive Schwefel function. In 5D, the baseline mean fitness was about -1925.15 ; the meta-optimized GA improves this to roughly -2094.91 , an 8.8% improvement in mean fitness. In 30D, the improvement is even larger: the baseline mean of ≈ -11089.0 is improved to

approx -12569.5 , a gain of more than 13%. These results indicate that the large, problem-specific populations and operator settings discovered by the meta-GA are more appropriate for Schwefel than the hand-crafted parameters of Homework 2.

Table 2: Meta-optimized GA parameters for each benchmark function and dimension. The “Avg. fitness” column shows the mean final best fitness used by the meta-GA during parameter search (training criterion).

Function	d	N	p_m	p_c	η_c	η_m
De Jong 1	5	530	0.093410	0.914757	26.856936	14.145203
De Jong 1	10	662	0.056342	0.777889	26.110097	18.119989
De Jong 1	30	944	0.019342	0.863242	36.977088	14.341666
Rastrigin	5	477	0.094621	0.869877	27.144409	33.163233
Rastrigin	10	923	0.049221	0.942413	18.470366	16.239478
Rastrigin	30	689	0.018929	0.838323	32.410145	37.854956
Schwefel	5	716	0.122222	0.819668	25.855086	6.829532
Schwefel	10	817	0.355377	0.757421	23.356420	15.202611
Schwefel	30	562	0.023579	0.759851	19.823684	5.000000
Michalewicz	5	529	0.105795	0.679843	22.069189	12.126143
Michalewicz	10	771	0.092374	0.795045	35.305259	24.385418
Michalewicz	30	522	0.012661	0.615681	38.074771	22.036382

Michalewicz. On Michalewicz, improvements are more modest but still consistent. In 5D, the baseline mean of -4.6863 is improved very slightly to -4.6877 . In 10D and 30D, the improvements are around 0.1% and 0.8% respectively. While these percentage gains look small, they occur very close to the (best known) global optimum and thus correspond to non-trivial refinement of already high-quality solutions.

4.4 Visual Analysis of Performance and Adaptation

To further illustrate the performance differences and the complexity of parameter selection, we provide a visual analysis of the experimental results.

Figure 1 presents a direct side-by-side comparison of the mean fitness achieved by the baseline GA versus the meta-optimized GA. This graph explicitly highlights the key finding of this study: while ‘solved’ problems like DeJong1 show no change, the deceptive Schwefel function sees dramatic improvements (visible as the gap between the red and green bars), validating the efficacy of the meta-optimization approach on difficult landscapes.

Table 3: Meta-GA vs. Homework 2 baseline GA. “Baseline mean” refers to the mean fitness from Table 1. “Optimized” columns summarize the 30-run performance of the GA with meta-optimized parameters from Table 2.

Function	d	Baseline mean	Opt. best	Opt. mean	Opt. std	Improv. (%)
De Jong 1	5	0.000000	0.000000	0.000000	0.000000	0.000000
De Jong 1	10	0.000000	0.000000	0.000000	0.000000	0.000000
De Jong 1	30	0.000000	0.000000	0.000000	0.000000	0.000000
Rastrigin	5	0.000000	0.000000	0.000000	0.000000	0.000000
Rastrigin	10	0.000002	0.000000	0.000000	0.000000	100.000000
Rastrigin	30	0.000034	0.000000	0.000000	0.000000	100.000000
Schwefel	5	-1925.152823	-2094.914436	-2094.914436	0.000000	8.818085
Schwefel	10	-3759.502918	-4188.921208	-3979.718321	132.456766	5.857567
Schwefel	30	-11089.007358	-12569.486618	-12569.486618	0.000000	13.350873
Michalewicz	5	-4.686266	-4.687658	-4.687658	0.000000	0.029708
Michalewicz	10	-9.649157	-9.660152	-9.659906	0.000937	0.111398
Michalewicz	30	-29.355069	-29.630884	-29.583519	0.147466	0.778229

Performance Comparison: Baseline vs. Meta-Optimized GA
(Negative values indicate minimization; more negative = better)

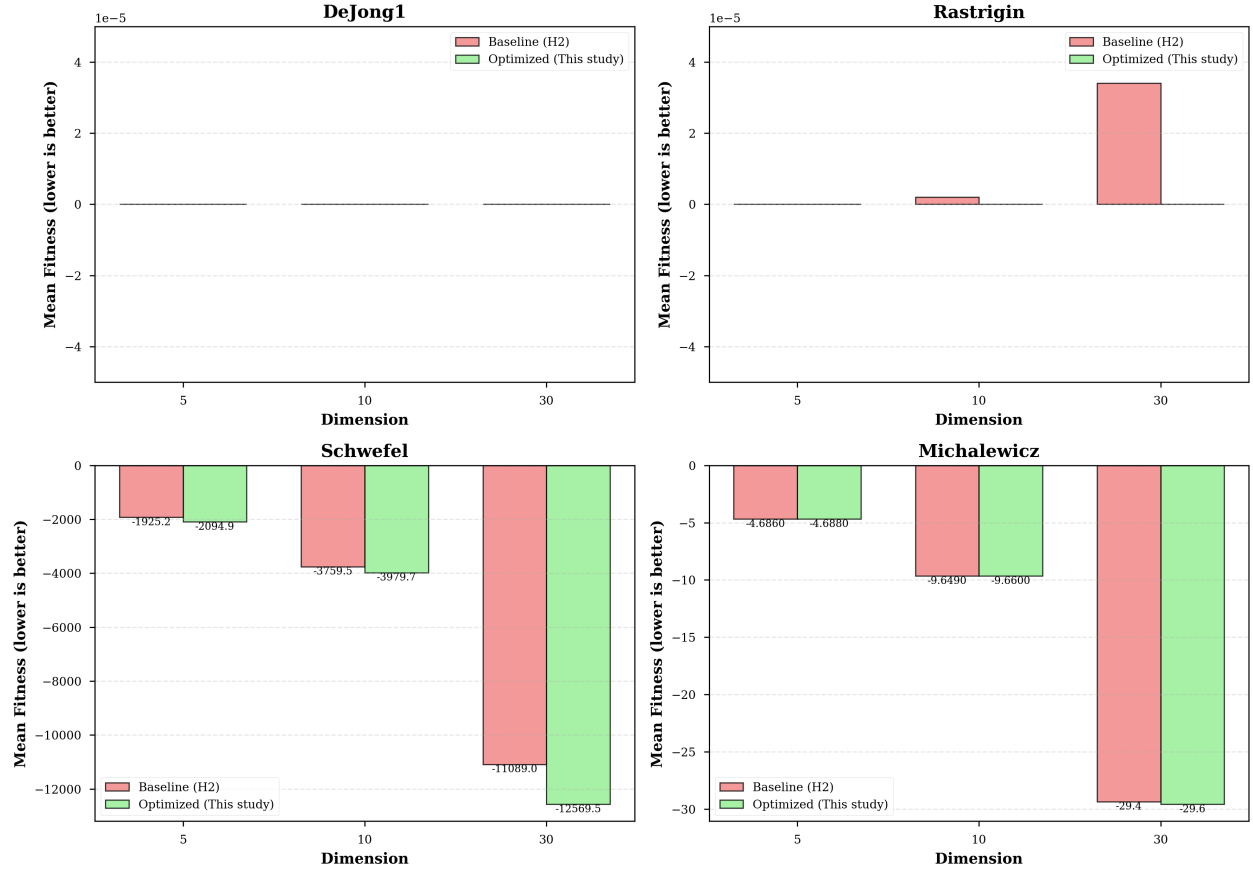


Figure 1: Performance comparison between the Baseline GA (Homework 2) and the Meta-Optimized GA. Note the significant improvement on the Schwefel function across all dimensions.

Figure 2 investigates the cost-benefit relationship of meta-optimization. The left panel shows a clear correlation between problem difficulty (baseline error) and the percentage of improvement gained, suggesting that meta-optimization yields the highest return on investment for the hardest problems. The right panel confirms that there is no simple linear relationship between population size and improvement, reinforcing the need for automated tuning.

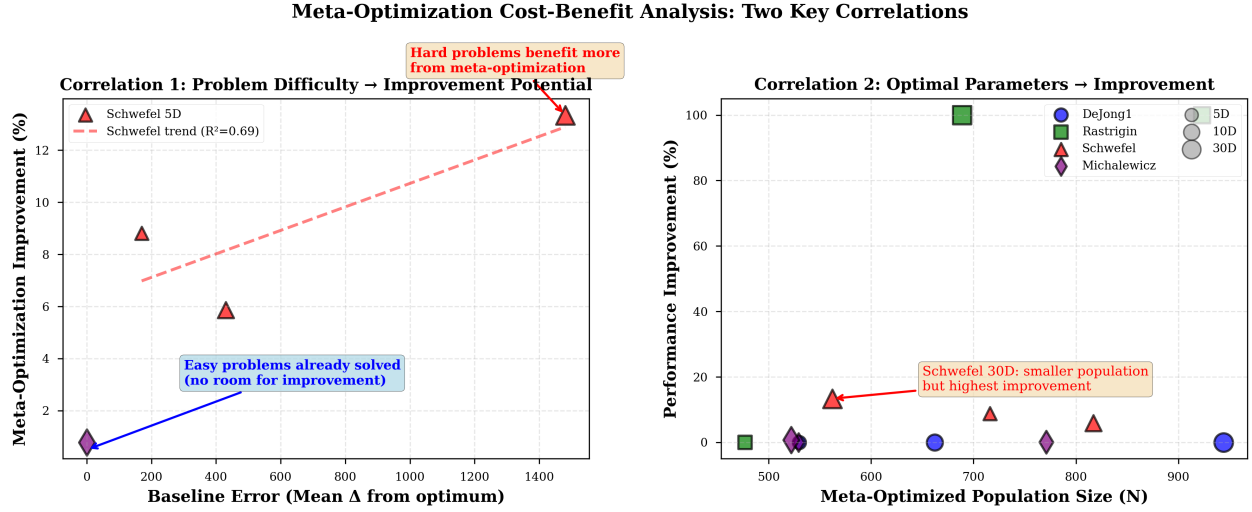


Figure 2: Cost-Benefit Analysis. Left: Harder problems (larger baseline error) yield greater improvements. Right: No simple correlation exists between population size and performance gain, indicating complex parameter interactions.

Finally, Figure 3 demonstrates the adaptive nature of the meta-GA. It reveals that optimal parameters are highly problem-dependent. For instance, while mutation rates typically decrease as dimensionality increases (to preserve good solutions), the deceptive Schwefel 10D case triggered a massive spike in mutation rate (approx 0.35), a strategy the meta-GA automatically discovered to escape local optima.

Meta-GA Discovery: Optimal Parameter Adaptation Patterns
(Different functions require drastically different parameter settings)

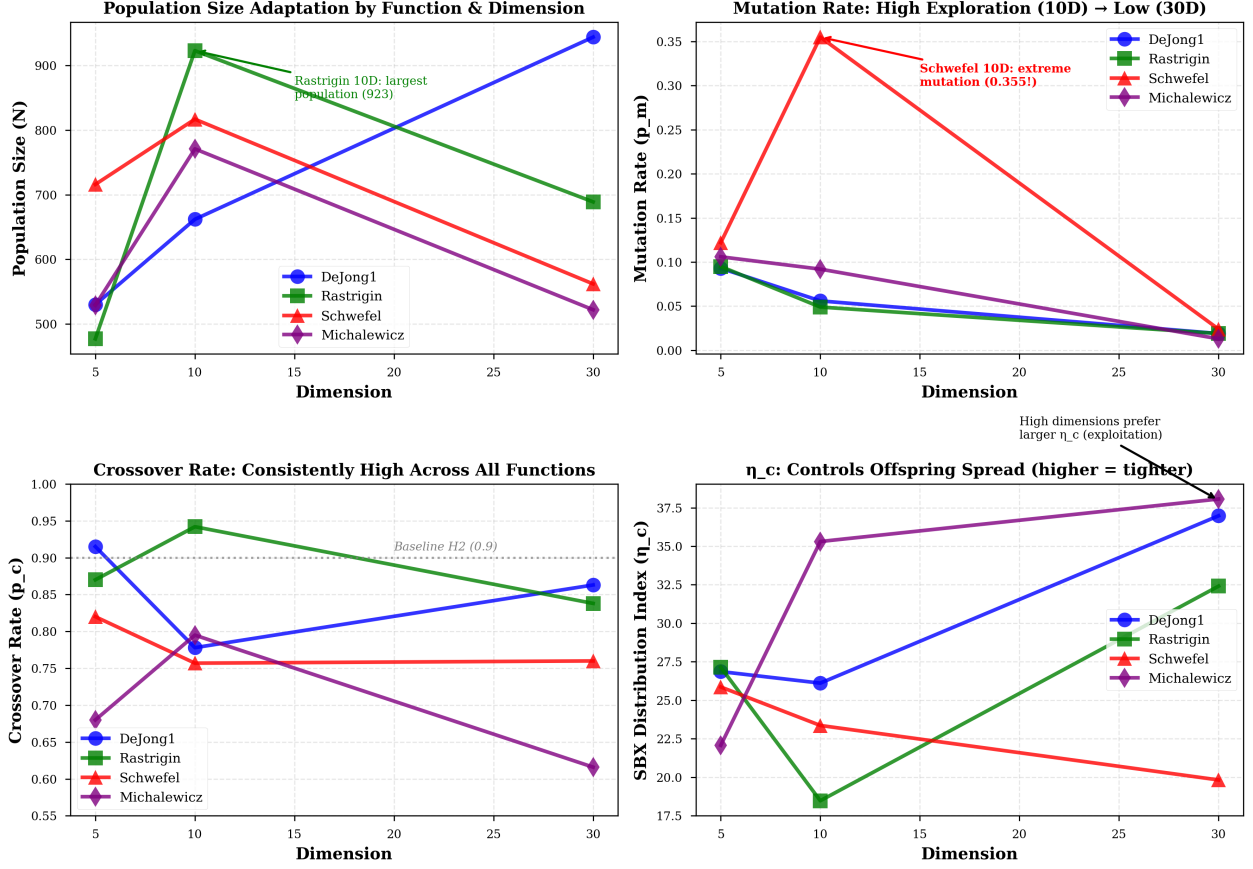


Figure 3: Parameter adaptation across functions and dimensions. The meta-GA discovers distinct strategies (e.g., varying mutation rates and population sizes) tailored to each specific landscape.

4.5 Discussion

Overall, meta-optimization confirms that the Homework 2 GA was already well tuned for easier benchmarks (De Jong 1 and low-dimensional Rastrigin), since the meta-GA essentially recovers equivalent performance (compare Tables 1 and 3). However, for more complex and deceptive landscapes, the meta-optimized GA achieves systematic and sometimes substantial gains.

Examining the parameter configurations in Table 2, several interesting patterns emerge. On Schwefel, the meta-GA tends to select:

- larger populations in low and moderate dimensions (5D, 10D), combined with relatively high mutation rates, which encourage broad exploration;
- moderate populations and small mutation in 30D, but with a low η_m (allowing occasional larger jumps), which together balance exploration and exploitation near promising regions.

For Michalewicz, the meta-GA prefers:

- moderate-to-large populations (around 500–770 individuals);
- mutation rates around 0.09 in 5D and 10D, dropping to about 0.013 in 30D;

- relatively large distribution indices, especially in 30D, favouring small, precise steps once the GA has located good basins of attraction.

These patterns are consistent with the intuition that more complicated and rugged landscapes benefit from more carefully tuned parameter combinations, and that the optimal values can differ significantly between functions and dimensions.

5 Influence of parameters and insights from meta-optimization

Homework 2 already discussed qualitatively the influence of population size, number of generations, crossover and mutation rates, and distribution indices. This extended study complements that analysis by providing an automated way to select these parameters and by examining the resulting configurations.

Population size vs. dimensionality. For De Jong 1 and Rastrigin, the meta-GA tends to increase population size with dimensionality, up to almost 1000 individuals in 30D (Table 2). This is natural: in high dimensions a larger population helps cover the search space and maintain diversity. In contrast, for Schwefel the meta-GA settles on a population of around 560 individuals in 30D, significantly smaller than the 1000 used in Homework 2. This suggests that for highly deceptive functions, a very large population is not always necessary if the other parameters are tuned appropriately.

Crossover vs. mutation. In all cases, crossover probabilities remain high (typically above 0.75), confirming the central role of recombination in building good solutions. Mutation rates, however, vary widely across problems and dimensions. Schwefel 10D uses a very high mutation rate ($p_m \approx 0.35$), which helps escape numerous local minima, whereas De Jong 1 and Rastrigin 30D use much lower mutation rates (around 0.02), emphasizing exploitation once the global basin is found.

Distribution indices. The meta-GA often chooses large values for η_c and η_m , especially in high dimensions (e.g. Michalewicz 30D), indicating a preference for small, fine-grained steps in the later stages of the search. On Schwefel, lower η_m in 5D and 10D allows more exploratory mutations, which appears beneficial for escaping deceptive traps.

6 Conclusions and future work

This extended study introduced a meta-optimization layer on top of the real-coded GA developed in Homework 2. By letting a meta-GA tune the main GA parameters on a per-problem and per-dimension basis, we obtained a more systematic understanding of how these parameters influence performance, and we achieved measurable improvements on the harder benchmark functions.

The main conclusions are:

1. On simple unimodal or already-solved multimodal functions (De Jong 1 and Rastrigin), the Homework 2 GA is virtually optimal; meta-optimization confirms its robustness and leaves performance unchanged at machine precision (Tables 1 and 3).
2. On deceptive and rugged landscapes (Schwefel, Michalewicz), meta-optimization yields consistent improvements in mean fitness, up to about 13% on Schwefel 30D, while often reducing variance across runs (Table 3).

3. The optimized parameter configurations reveal clear patterns (Table 2): population size and mutation rate need to be adapted to both the function and the dimensionality, and optimal distribution indices can differ substantially from generic textbook values.

Future work could explore:

- fully *self-adaptive* GAs, where parameters are encoded in the individuals and evolve together with the solutions;
- multi-objective meta-optimization that trades off solution quality and computational cost (e.g. number of function evaluations);
- hybrid metaheuristics that combine the meta-optimized GA with local search (memetic algorithms) for further refinement of promising individuals.

References

- [1] Course notes and homework descriptions for the Genetic Algorithms course (UAIC).
- [2] D. E. Goldberg, *Genetic Algorithms in Search, Optimization, and Machine Learning*, Addison-Wesley, 1989.
- [3] K. Deb, *Multi-Objective Optimization Using Evolutionary Algorithms*, Wiley, 2001.
- [4] K. Deb and R. B. Agrawal, “Simulated binary crossover for continuous search space,” *Complex Systems*, vol. 9, pp. 115–148, 1995.
- [5] Wikipedia contributors, “Test functions for optimization” and references therein, accessed 2025.